Step 1 — ContentForge, Reachable Over HTTP for the First Time — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — The front door, not a new agent
- Hint 2 — The error paths, and testing without paying

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — The front door, not a new agent

Nothing about the five agents changes in this step. You're adding a **front door**: a FastAPI app with a route that takes a topic as JSON, runs Project 4's graph, and returns the result as JSON.

The pieces:

- `app = FastAPI()` and a `/health` route — the smallest thing that answers.
- A Pydantic model for the request body: `topic: str`.
- `POST /run-task`: build the graph, `graph.invoke(start_state(topic))`, return the draft and whether it was accepted.
- `load_config()` before the app is built — it reads `.env`, so the agents can find the key.

Run it with `uvicorn src.api.main:app --reload` and try it at `http://localhost:8000/docs`.

Split it the way Doc12's Build Task does, plus one extra file:

- `pipeline.py` → `run_pipeline(topic) -> dict` — the **one** door into the team. It runs the graph and returns plain values: `article` (the draft), `accepted`, `reason` (the last routing entry's reason) and `steps` (one text line per routing entry).
- `api/schemas.py` → `TaskRequest(topic)` with `Field(min_length=1, max_length=200)`, and `TaskResponse(accepted, reason, article)`.
- `api/routes.py` → an `APIRouter` with `/health` and `/run-task`.
- `api/main.py` → `load_config()`, `FastAPI()`, `include_router(router)`.

Why a separate `run_pipeline()`: the route never touches the graph's state, and a test can replace this one function with a fake.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The error paths, and testing without paying

Two things can go wrong that the client should hear about differently:

- **Bad input** — a topic of `" "`. Pydantic lets it through (it *is* a string), so check `topic.strip()` yourself and raise a **400**.
- **A crash inside the team** — a bad key, a timeout. Catch it, log the full error on the server, and return a **500** with one plain sentence. Never `str(exc)` — it can hold file paths or secrets.

For tests, don't run the real team: it's slow, it costs money, and the article changes every time.

```
routes.py, POST /run-task:
    logger = get_logger(__name__)
    if not request.topic.strip(): raise 400 "topic cannot be empty"
    try:
        outcome = run_pipeline(request.topic)
    except Exception:
        logger.exception("run-task failed")
        raise 500 INTERNAL_ERROR
    return TaskResponse(accepted=..., reason=..., article=...)
```

`tests/api/test_routes.py` — pytest, `TestClient(app)`, and a fake team:

```
def use_fake_team(monkeypatch, fake) -> None:
    # swaps routes.run_pipeline for the fake, for this test only
    monkeypatch.setattr(routes, "run_pipeline", fake)
```

One fake returns a fixed dict; another raises `RuntimeError`. Then one test per case: `/health` is 200, a valid topic is 200, `{}` is 422, `" "` is 400, a crash is a plain 500. Run with `pytest tests/api -v` — Project 4's print-style tests aren't pytest tests yet (Step 4).

Full solution: Show me the solution